Frontend Documentation — Bhealth Insurance (Lumina CRM)

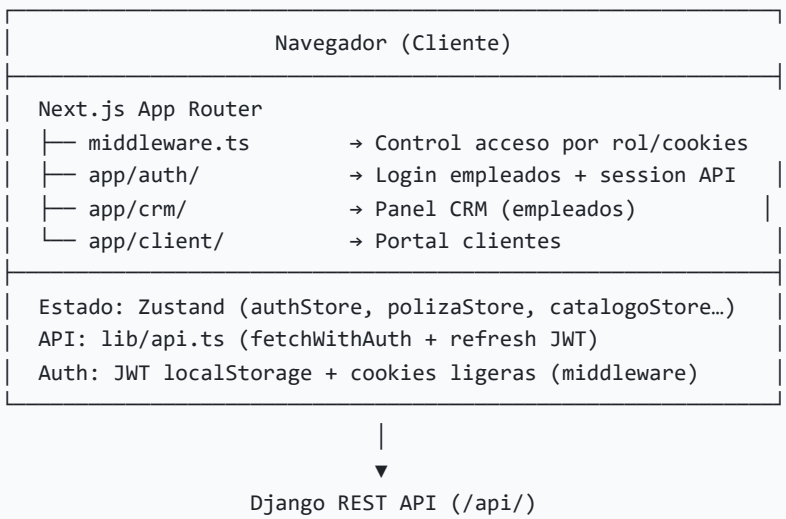
Documentación técnica del frontend **Next.js** del sistema de gestión de seguros de salud BHealth Insurance / Lumina CRM.

Stack: Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS 4 · Zustand
Backend API: Django REST en `http://127.0.0.1:8000/api` (desarrollo)

Tabla de Contenidos

1. Arquitectura General
2. Estructura del Proyecto
3. Rutas de la Aplicación
4. Páginas Principales
5. Componentes
6. Estado Global (Zustand)
7. Capa API (`lib/api.ts`)
8. Autenticación y Seguridad
9. Permisos y Roles
10. Tema e Internacionalización
11. Variables de Entorno
12. Desarrollo Local
13. Despliegue en Producción
14. Flujos de Usuario
15. Módulo de Pólizas
16. Portal del Cliente
17. Apéndice

Arquitectura General



Principios de diseño

- **Separación estricta CRM / Portal:** el middleware impide que clientes entren al CRM y viceversa.
- **JWT en localStorage + cookies ligeras:** el token completo vive en `localStorage`; cookies `auth_session`, `user_role` y `portal_client` permiten al middleware validar sin leer el JWT en edge.
- **Refresh centralizado:** `fetchWithAuth` renueva el token en 401 con cola de peticiones pendientes.
- **Catálogos precargados:** `CatalogoProvider` carga agencias, agentes, planes, etc. al entrar al CRM.

Estructura del Proyecto

```
frontend/
├── app/                          # App Router (páginas)
│   ├── layout.tsx                # Root: ThemeProvider, AuthInit
│   ├── page.tsx                  # / → redirect login
│   ├── auth/
│   │   ├── login/page.tsx        # Login empleados
│   │   ├── register/page.tsx     # Registro empleados
│   │   └── session/route.ts      # API route cookies servidor
│   ├── crm/                      # Panel CRM
│   │   ├── layout.tsx            # Sidebar, theme, catálogos
│   │   ├── dashboard/
│   │   ├── polizas/
│   │   ├── cotizaciones/
│   │   ├── users/
│   │   └── configuraciones/
│   └── client/                   # Portal clientes
│       ├── page.tsx              # Login/registro cliente
│       └── dashboard/
├── components/
│   ├── ui/                       # Primitivos shadcn/Radix (~50)
│   ├── polizas/                  # Módulo pólizas (~60 archivos)
│   ├── client/                   # Componentes portal
│   ├── sidebar/                  # Navegación CRM/cliente
│   ├── auth/                     # Login, guards
│   ├── configuraciones/          # Settings CRM
│   └── providers/                # AuthInit, CatalogoProvider
├── lib/
│   ├── api.ts                    # Cliente API principal
│   ├── authSession.ts            # Cookies y persistencia
│   ├── jwt.ts                    # Decode JWT, roles
│   ├── middlewareAccess.ts       # Lógica redirect middleware
│   ├── roles.ts                  # Roles CRM
│   ├── permissions.ts            # Permisos UI
│   └── familyMemberUtils.ts      # Utilidades familiares
├── store/                        # Zustand stores
├── hooks/                        # Custom hooks
├── types/                        # TypeScript types
└── middleware.ts                 # Edge middleware
```

Rutas de la Aplicación

Mapa de rutas

Ruta	Zona	Acceso	Descripción
`/`	—	Público	Redirect → `/auth/login`
`/auth/login`	Auth	Público*	Login empleados CRM
`/auth/register`	Auth	Público	Registro empleados
`/crm`	CRM	Empleados	Landing legacy (sin sidebar)
`/crm/dashboard`	CRM	Empleados	Dashboard analítico
`/crm/users`	CRM	Admin/Manager	Gestión usuarios
`/crm/users/[id]`	CRM	Admin/Manager	Detalle usuario
`/crm/polizas`	CRM	Empleados	Lista pólizas
`/crm/polizas/[id]`	CRM	Empleados	Detalle póliza
`/crm/polizas/nueva`	CRM	Empleados	Wizard nueva póliza
`/crm/polizas/export`	CRM	Empleados	Export CSV
`/crm/cotizaciones`	CRM	Empleados	Lista cotizaciones
`/crm/citas`	CRM	Empleados	Citas médicas
`/crm/formularios-de-firmas`	CRM	Empleados	Consentimiento electrónico
`/crm/analytics`	CRM	Manager	Comisiones
`/crm/configuraciones`	CRM	Empleados	Configuración CRM
`/client`	Portal	Público	Login/registro cliente
`/client/dashboard`	Portal	Clientes	Dashboard portal
`/client/[clientId]/profile`	Portal	Clientes	Perfil extendido
`/client/[clientId]/configuraciones`	Portal	Clientes	Preferencias

*Si ya hay sesión activa, redirige al dashboard correspondiente.

Middleware (`middleware.ts`)

Matcher: `['/', '/auth', '/crm/:path*', '/client/:path*', '/auth/login']`

Condición	Acción
Sin sesión + `/crm/*`	→ `/auth/login`
Cliente + `/crm/*`	→ `/client/dashboard`
Empleado + `/client/*` (protegido)	→ `/crm/dashboard`
Autenticado + `/auth/login`	→ dashboard según rol

Lógica en `lib/middlewareAccess.ts` usando cookies `auth_session`, `user_role`, `portal_client` y JWT opcional en cookie.

Páginas Principales

`/crm/dashboard`

Dashboard analítico del CRM:

- KPIs: total pólizas, aplicantes, ventas, retención
- Gráficos: barras por mes, pie por producto/aseguradora, mapa US
- Rankings de agentes
- Filtro por rango de fechas + botón "Ver todo"
- Fuente: `GET /api/dashboard/stats/`

`/crm/polizas`

Lista paginada de pólizas activas (excluye cotizaciones QUOTE):

- Filtros: estado, agente, fechas, búsqueda
- Acciones: ver, editar, copiar, archivar
- Botón "Nueva póliza" → modal o wizard
- Store: `usePolizaStore`

`/crm/polizas/[id]`

Página de detalle de póliza (la más compleja):

- Resumen y metadatos (`PolizaResumen` , `MetadatosPoliza`)
- Miembros de familia (`FamilyMembersSection`)
- Información de pago (`InfoPagoPoliza` , `PaymentMethodsSection`)
- Documentos (`DocumentosPoliza`)
- Notas (`NotasPoliza`)
- Recordatorios

- Cambios de vida (`CambiosDeVidaPanel`)
- Médicos y medicamentos (`MedicosMedicamentosSection`)
- Presencia/bloqueo edición (`PolizaPresenceBar` , `PolizaEditLockContext`)
- Timeline (`PolizaTimeline`)

`/crm/polizas/nueva`

Wizard multi-paso para crear póliza/cotización:

1. Información de póliza (tipo producto, fecha efectiva, agente)
2. Información personal (nuevo o cliente existente)
3. Grupo familiar (cónyuge, dependientes)
4. Dirección física (obligatoria; lookup ZIP → condado)
5. Envío → estado QUOTE en cotizaciones

`/crm/cotizaciones`

Misma UI que pólizas pero filtrada a estado `QUOTE` . Desde detalle se puede validar y enviar (`validar-envio` , `enviar`).

`/crm/configuraciones`

Paneles modulares:

- Información de agencia y branding
- Estados de póliza
- Etiquetas
- Notificaciones
- Importación Excel
- Usuarios de facturación

`/client/dashboard`

Portal del cliente autenticado:

- Datos personales (`MisDatosCard`)
- Mis pólizas (`MisPolizasCard`) — multi-póliza
- Grupo familiar
- Solicitudes de cambio (`SolicitudCambioModal`)
- Subida de documentos
- API: `portalClienteApi.getDashboard()`

Componentes

Organización por dominio

Carpeta	Contenido	Archivos clave
`components/polizas/`	Todo el módulo pólizas	`FamilyMembersSection`, `PolizaResumen`, `DocumentosPoliza`, `CambiosDeVidaPanel`
`components/client/`	Portal cliente	`MisPolizasCard`, `MisDatosCard`, `SolicitudCambioModal`
`components/sidebar/`	Navegación	`sidebarCRM.tsx`, `sidebarClient.tsx`
`components/auth/`	Autenticación	`loginUser.tsx`, `PermissionGuard.tsx`, `SplitLoginLayout.tsx`
`components/configuraciones/`	Settings	`AgenciaConfig`, `EtiquetasConfig`, `StatusList`
`components/ui/`	Primitivos	button, dialog, table, input, select, sidebar...
`components/providers/`	Contexto global	`AuthInit.tsx`, `CatalogoProvider.tsx`

Componentes críticos de pólizas

Componente	Responsabilidad
`FamilyMembersSection`	Lista/expandir miembros, ingresos, estatus migratorio, docs legales
`FamilyMemberFormModal`	Crear/editar miembro familiar
`PolizaEditLockContext`	Bloqueo edición cuando otro usuario edita
`PolizaPresenceBar`	Barra usuarios viendo la póliza
`CambiosDeVidaPanel`	Eventos de vida + log comparativo
`EditPlanDetailsModal`	Cobertura, plan, CMS ID, metal, costos
`DocumentosPoliza`	Carpetas, upload, etiquetas, estados documento
`InfoPagoPoliza`	Pago recurrente, estado pago, calendario primas

Estado Global (Zustand)

Export central: `store/index.ts`

Store	Archivo	Persistido	Responsabilidad
`useAuthStore`	`authStore.ts`	Sí (`auth-storage`)	Login, JWT, perfil, rol
`usePolizaStore`	`polizaStore.ts`	No	Lista/detalle pólizas, filtros, paginación
`useCatalogoStore`	`catalogoStore.ts`	Sí	Agencias, agentes, planes, estados
`useConfiguracionesCRMStore`	`configuracionesCRMStore.ts`	Sí	Settings agencia
`useClienteStore`	`clienteStore.ts`	No	Datos cliente
`useNotificationStore`	`notificationStore.ts`	No	Notificaciones in-app
`useLanguageStore`	`languageStore.ts`	Sí	Preferencia es/en
`useDashboardStore`	`dashboardStore.ts`	No	Stats dashboard
`useCRMUsersStore`	`crmUsersStore.ts`	No	Gestión usuarios
`useDocumentoStore`	`documentoStore.ts`	No	Documentos
`useTareaStore`	`tareaStore.ts`	No	Tareas
`useUIStore`	`uiStore.ts`	No	Sidebar, modales, tabs
`useFormulariosFirmasStore`	`formulariosFirmasStore.ts`	No	Consent requests

Auth store — ciclo de vida

1. `login()` → `POST /api/login/` → guarda tokens
2. `getProfile()` → `GET /api/profile/` → rol definitivo
3. `persist` → `localStorage` (`auth-storage` , `access_token`)
4. `setAuthCookies()` → cookies ligeras para middleware
5. `syncServerSessionCookies()` → `POST /auth/session` (Set-Cookie servidor)
6. Rehidratación: reconcilia localStorage ↔ cookies sin logout fantasma

---

Capa API (`lib/api.ts`)

Configuración base

```
// Desarrollo
API_BASE_URL = 'http://127.0.0.1:8000/api'

// Producción (sin NEXT_PUBLIC_API_URL)
API_BASE_URL = '/api'  // Nginx proxy same-origin
```

Infraestructura

Export	Función
`fetchWithAuth`	Bearer + refresh 401 + rate limiter
`refreshAccessTokenShared`	Refresh deduplicado
`createCrudApi`	CRUD genérico REST
`normalizeApiList`	Normaliza `{ results: [] }` DRF
`resolveApiMediaUrl`	URLs de archivos media
`clearClientAuthStorage`	Limpieza total auth
`armAuthGracePeriod`	Evita logout tras login (15s)

Módulos API

Módulo	Endpoints backend
`authApi`	login, refresh, register, profile, logout
`polizasApi`	CRUD + cotizaciones, enviar, presence, archive
`clientesApi`	CRUD + verificar-duplicado, contexto-poliza
`agentesApi`	CRUD + fetchAllPages, listRegistrados
`portalClienteApi`	dashboard, solicitar-cambio, documentos
`cambiosDeVidaApi`	CRUD cambios de vida
`documentosApi`	CRUD + preview
`consentRequestsApi`	Firma electrónica
`notificacionesApi`	Notificaciones usuario
`polizasFlatApi`	Import/export flat

Uso recomendado:

```
import api from '@/lib/api';

const polizas = await api.polizas.listPaginated({ page: 1 });
const perfil = await authApi.getProfile();
```

> **Nota:** `services/api.ts` es legacy — usar siempre `lib/api.ts`.

Autenticación y Seguridad

Flujo de login empleado

```
Usuario → /auth/login
  → authStore.login()
  → POST /api/login/ (access + refresh)
  → getProfile() (rol + departamento)
  → canUseEmployeeLogin() (rechaza clientes)
  → POST /auth/session (cookies servidor)
  → window.location.replace('/crm/dashboard')
  → middleware valida cookies en /crm/*
  → CRM layout inicia employeeTokenRefresh
```

Flujo de login cliente

```
Usuario → /client
  → authStore.login()
  → canUseEmployeeLogin() === false (es cliente)
  → syncServerSessionCookies (portal_client=1)
  → redirect /client/dashboard
  → portalClienteApi.getDashboard()
```

Almacenamiento de tokens

Ubicación	Contenido
`localStorage.auth-storage`	Estado Zustand persistido
`localStorage.access_token`	JWT access (directo)
`localStorage.refresh_token`	JWT refresh
Cookie `auth_session=1`	Hint sesión activa
Cookie `user_role`	Rol para middleware
Cookie `portal_client=1`	Portal cliente
Cookie `access_token`	JWT opcional (si ≤3800 chars)

Refresh automático

- **Reactivo:** `fetchWithAuth` intercepta 401, renueva token, reintenta cola
- **Proactivo:** `employeeTokenRefresh.ts` renueva 1 min antes de expirar (solo CRM layout)
- **Grace period:** 15s tras login para evitar race conditions

API route `/auth/session`

Método	Acción
POST	Establece cookies vía `Set-Cookie` (más fiable que `document.cookie`)
DELETE	Borra todas las cookies de sesión

Permisos y Roles

Roles CRM (frontend `lib/roles.ts`)

`super_admin`, `led`, `client_support`, `gestion_resolucion`, `health_sherpa`, `agente`, `cobranza`, `recuperacion`, `recepcion`, `soporte_cliente`

Helpers

Función	Uso
`hasCRMAccess(role)`	¿Puede entrar al CRM?
`canManageUsers(role)`	Gestión usuarios
`canAccessClientPortal(...)`	¿Es portal cliente?
`usePermission()` hook	Guard en componentes
`PermissionGuard`	Wrapper condicional UI

Tema e Internacionalización

Tema (next-themes)

- Configurado en `app/layout.tsx`: `attribute="class"`, `defaultTheme="light"`
- Toggle en header CRM (icono Sol/Luna)
- Clases Tailwind: `dark:bg-[#1e293b]`, `dark:text-white`, etc.

Idioma

- UI principalmente en **español** (strings hardcoded)
- `useLanguageStore`: persiste `es` | `en`
- Toggle ES/EN en header CRM (infraestructura; traducción global no implementada con i18n framework)

Variables de Entorno

Crear `frontend/.env.local` (no commitear):

```
NEXT_PUBLIC_SITE_URL=http://localhost:3000
NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api
NEXT_PUBLIC_API_MIN_INTERVAL_MS=20
```

Producción (`frontend/.env.production`)

```
NEXT_PUBLIC_SITE_URL=http://2.25.68.83
# NEXT_PUBLIC_API_URL=/api
```

Variable	Requerida	Default dev	Descripción
`NEXT_PUBLIC_API_URL`	No	`http://127.0.0.1:8000/api`	URL base API
`NEXT_PUBLIC_SITE_URL`	Prod	—	URL pública (evita redirect localhost en VPS)
`NEXT_PUBLIC_API_MIN_INTERVAL_MS`	No	`20`	Rate limit entre requests

Desarrollo Local

Requisitos

- Node.js 20+
- Backend Django en `http://127.0.0.1:8000`

Pasos

```
# Terminal 1 — Backend
cd backend
cp .env.local.example .env    # si no existe
python manage.py migrate
python manage.py ensure_crm_admin
python manage.py runserver

# Terminal 2 — Frontend
cd frontend
cp .env.local.example .env.local
npm install
npm run dev
```

Abrir http://localhost:3000 → redirect a `/auth/login`.

Credenciales demo: `lumina-admin` / `admin123` (si `ensure_crm_admin` ejecutado).

Scripts npm

Script	Descripción
`npm run dev`	Servidor desarrollo (:3000)
`npm run build`	Build producción
`npm run start`	Servir build
`npm run lint`	ESLint
`npm run verify:client-login`	Smoke test login cliente

Despliegue en Producción

VPS (Hostinger + Nginx)

1. `npm ci && npm run build` en `frontend/`
2. `NEXT_PUBLIC_SITE_URL` = IP o dominio público
3. Nginx proxy `/` → Next.js (:3000), `/api/` → Django (:8000)
4. Servicio systemd: `lumina-frontend.service`
5. Script: `deploy/scripts/update-app.sh`

Consideraciones

- `NEXT_PUBLIC_*` se embeben en build — rebuild tras cambiar
- Middleware usa `NEXT_PUBLIC_SITE_URL` para redirects correctos (no `localhost`)
- Cookies de sesión duran 7 días (independiente del JWT de 60 min)

Flujos de Usuario

A. Login empleado CRM

1. `/auth/login` → credenciales
2. JWT + perfil + cookies
3. `/crm/dashboard`
4. Sidebar carga catálogos

B. Crear cotización/póliza

1. `/crm/polizas/nueva` (wizard) o modal rápido
2. Tipo producto + agente + fecha efectiva

3. Cliente nuevo o existente (`ClienteCombobox`)
4. Grupo familiar + dirección física
5. POST → estado QUOTE
6. Aparece en `/crm/cotizaciones`
7. En detalle: completar plan → `validar-envio` → `enviar` → póliza activa

C. Editar póliza

1. `/crm/polizas/[id]`
2. `POST presence` modo `edit` (bloqueo)
3. Editar secciones (familia, pago, docs, notas, cambios vida)
4. Heartbeat mantiene presencia
5. Al salir: liberar lock

D. Portal cliente

1. `/client` → login
2. `/client/dashboard`
3. Ver pólizas, solicitar cambios, subir documentos
4. PATCH/DELETE solicitudes pendientes propias

E. Dashboard analítico

1. `/crm/dashboard`
2. KPIs globales + filtro periodo
3. Gráficos Recharts + mapa `react-simple-maps`
4. Años seleccionados persistidos en perfil usuario

Módulo de Pólizas

Estados de póliza relevantes

Estado	Dónde aparece
`QUOTE`	`/crm/cotizaciones`
Activos (no QUOTE)	`/crm/polizas`
Archivadas	Filtro en lista

Flujo departamental (UI)

Acciones en detalle según `flujo_etapa`:

- Pase a Manager
- Pase a Sherpa
- Devolver a Asistente (requiere nota)
- Cerrar en Sherpa

Eventos de vida (`CambiosDeVidaPanel`)

Tipos: nueva dirección, cambio ingresos, estatus migratorio, nuevo miembro, embarazo, fallecido, otros.

Estados: `PENDIENTE`, `EN_PROCESO`, `COMPLETADO`.

Documentos

Estados: `PENDIENTE`, `PROCESADO`, `RECHAZADO`, `COMPLETADO`, `EXPIRADO`.

Categorías: `CARTA_MERCADO`, `ASEGURANCIA`, `CONSENTIMIENTO`, etc.

Pagos

Estados pago: `PENDIENTE`, `PAGO_AUTOMATICO`, `FALLIDO`, `PAGADO`, `NO_APLICABLE`.

Portal del Cliente

API (`portalClienteApi`)

Método	Función frontend
`getDashboard(polizaId?)`	Carga dashboard
`solicitarCambio(data)`	Solicitud cambio datos
`solicitarDocumento(formData)`	Upload documento
`editarSolicitud(id, data)`	PATCH solicitud
`cancelarSolicitud(id)`	DELETE solicitud

Componentes dashboard

Componente	Datos
`MisDatosCard`	Info personal cliente
`MisPolizasCard`	Selector multi-póliza
`GrupoFamiliarCard`	Miembros familia
`SolicitudCambioModal`	Formulario cambio
`DocumentoUploadCard`	Subida documentos

Apéndice

Dependencias principales

Paquete	Uso
`next` 16	Framework
`react` 19	UI
`zustand`	Estado
`react-hook-form` + `zod`	Formularios
`@radix-ui/*`	Primitivos accesibles
`recharts`	Gráficos dashboard
`react-simple-maps`	Mapa US
`next-themes`	Dark mode
`lucide-react`	Iconos
`xlsx`	Excel import/export
`date-fns`	Fechas

Path alias

```
"@/*": ["./*"]
```

Archivos de configuración local

Archivo	Propósito
`frontend/.env.local`	Variables desarrollo
`frontend/.env.local.example`	Plantilla
`frontend/.env.production`	Variables VPS (no commitear)
`backend/.env`	Variables backend local

Documentación relacionada

Archivo	Contenido
`GUIA_EMPLEADO_CRM.md`	Manual para empleados (sin conocimientos técnicos)
`API_CRUD_DETAILED.md`	Backend API completa
`FRONTEND_DOC.md`	Frontend Next.js
`VPS_DEPLOY_UBUNTU.md`	Despliegue completo en VPS Ubuntu
`roles.md`	Jerarquía roles y permisos
`deploy/README.md`	Plantillas y referencia rápida deploy
`frontend/store/README.md`	Stores Zustand

Puntos de entrada para nuevos desarrolladores

1. Leer este documento + `API_CRUD_DETAILED.md`
2. Levantar backend + frontend local
3. Login `lumina-admin` → explorar `/crm/polizas/[id]`
4. Revisar `lib/api.ts` y `store/authStore.ts`
5. Entender `middleware.ts` antes de cambiar rutas

Última actualización: Julio 2026
Versión frontend: Next.js 16 / React 19
Repositorio: Bhealth-Insurance (Lumina CRM)